Persisting Entities

Persisting Entities

This section describes how to persist (save) entities with Spring Data JPA.

Saving Entities

Saving an entity can be performed with the `CrudRepository.save(…)` method. It persists or merges the given entity by using the underlying JPA `EntityManager`. If the entity has not yet been persisted, Spring Data JPA saves the entity with a call to the `entityManager.persist(…)` method. Otherwise, it calls the `entityManager.merge(…)` method.

Entity State-detection Strategies

Spring Data JPA offers the following strategies to detect whether an entity is new or not:

1. Version-Property and Id-Property inspection (**default**): By default Spring Data JPA inspects first if there is a Version-property of non-primitive type. If there is, the entity is considered new if the value of that property is `null`. Without such a Version-property Spring Data JPA inspects the identifier property of the given entity. If the identifier property is `null`, then the entity is assumed to be new. Otherwise, it is assumed to be not new. In contrast to other Spring Data modules, JPA considers `0` (zero) as the first inserted version of an entity and therefore, a primitive version property cannot be used to determine whether an entity is new or not.
2. Implementing `Persistable`: If an entity implements `Persistable`, Spring Data JPA delegates the new detection to the `isNew(…)` method of the entity. See the JavaDoc for details.
3. Implementing `EntityInformation`: You can customize the `EntityInformation` abstraction used in the `SimpleJpaRepository` implementation by creating a subclass of `JpaRepositoryFactory` and overriding the `getEntityInformation(…)` method accordingly. You then have to register the

custom implementation of `JpaRepositoryFactory` as a Spring bean. Note that this should be rarely necessary. See the JavaDoc for details.

Option 1 is not an option for entities that use manually assigned identifiers and no version attribute as with those the identifier will always be non-`null`. A common pattern in that scenario is to use a common base class with a transient flag defaulting to indicate a new instance and using JPA lifecycle callbacks to flip that flag on persistence operations:

Example 1. A base class for entities with manually assigned identifiers

JAVA

```
@MappedSuperclass
public abstract class AbstractEntity<ID> implements Persistable<ID> {

  @Transient
  private boolean isNew = true;  1

  @Override
  public boolean isNew() {
    return isNew;  2
  }

  @PostPersist  3
  @PostLoad
  void markNotNew() {
    this.isNew = false;
  }

  // More code…
}
```

1. Declare a flag to hold the new state. Transient so that it's not persisted to the database.
2. Return the flag in the implementation of `Persistable.isNew()` so that Spring Data repositories know whether to call `EntityManager.persist()` or `….merge()`.
3. Declare a method using JPA entity callbacks so that the flag is switched to indicate an existing entity after a repository call to `save(…)` or an instance creation by the persistence provider.